

The Prompt Does Not Contain Its Own Force

Distributed Specification, Commitment, and Repair in Language-Controlled Systems

Watson Hartsoe

Digital Media, Georgia Institute of Technology

Working Paper · 21 September 2026

Abstract

The prompt is usually treated as a piece of text supplied to a model. That unit is too small for many of the systems now described as prompt-driven. A command such as “Create a blue square there” can distribute its specification across speech, gesture, defaults, persistent state, and an executor. The sentence alone does not contain the location, the default size, the authority to act, or the test by which the result becomes acceptable. This paper develops a different account from a set of concrete technical cases: Bolt’s *Put-That-There*, McCarthy’s LISP evaluator, Winograd’s SHRDLU, Sutherland’s Sketchpad, ReAct-style language agents, Pfungst’s Clever Hans experiments, Ashby’s control theory, Naur’s theory-building account of programming, and program sketching. Across these cases, three recurrent operations become visible. Specification is distributed across channels and state. Operative force appears only when a representation crosses a commitment boundary. Correspondence is established through feedback and repair rather than by textual completeness alone. The resulting claim is not that all prompting has one hidden essence. It is that prompt research should stop attributing the work of the whole arrangement to the visible string. The practical consequence is an instrumentation agenda: track parameter provenance, expose consequential bindings before commitment, distinguish state spaces, evaluate trajectories as well as terminal outputs, and treat repair cost as a first-class property of prompt systems.

Keywords: prompting; language-games; multimodal interfaces; execution; repair; distributed specification; agent systems; human-computer interaction

1 Introduction

A blue square appears on a screen. The user has said, “Create a blue square there.” It is tempting to describe the resulting object as the execution of that sentence. Richard Bolt’s *Put-That-There* makes this description difficult to sustain. In Bolt’s system, the words supply the color and shape, the spoken “there” samples a pointing stream to supply a location, and the size can be supplied by a system default rather than by the user’s utterance [Bolt, 1980]. The rendered square is one object, but its properties do not have one source. The



event that produces it is already distributed across language, gesture, timing, interface state, and a procedure that knows how to combine them.

This small case exposes a larger problem with the ordinary unit of prompt analysis. Current discussion often treats a prompt as an inscription whose quality can be judged by inspecting the words. Yet the words can be only one contribution to an operative arrangement. A referent can be supplied by pointing rather than naming. A parameter can be supplied by a default rather than by the user. A pronoun can depend on remembered action state. A string can remain inert or become executable depending on an evaluator or router outside the string. A model-generated “thought” can leave the external environment unchanged while still altering the context from which a later action is chosen [McCarthy, 1960, Winograd, 1971, Yao et al., 2023]. These are not marginal complications. They change what counts as the thing being studied.

The argument of this paper is that the operative unit of prompting is often a distributed event rather than a text object. Three distinctions follow. First, specification has provenance: different properties of an outcome can be supplied by different channels, stored states, defaults, or components. Second, interpretation and action are separated by commitment boundaries: a representation becomes consequential only when some part of a system binds, routes, authorizes, or executes it. Third, correctness is often interactional: users discover what the system took them to mean through feedback, and effective performance depends on the cost and timing of repair as much as on first-pass recognition. These distinctions do not define a timeless essence of “the prompt.” They provide a way to locate where the work happens in concrete systems.

The point is methodological as much as conceptual. When prompt research attributes success or failure to the text alone, it can misidentify the cause. A successful output can depend on a hidden default. A benchmark score can depend on information in the evaluator’s side of an arrangement. A final answer can be correct even when the process that produced it is defective. A supervisor can possess formal authority while lacking enough observable dimensions and interventions to regulate the system in practice [Pfungst, 1911, Solar-Lezama et al., 2007, Ashby, 1956]. The research task is therefore to decompose the apparent prompt-response pair into the state transitions that make it operative.

2 From Prompt String to Distributed Specification

Bolt’s interface is useful because its machinery is explicit enough to make specification provenance visible. The utterance “Create a blue square there” has the surface form of a single instruction, but the system does not obtain all of its arguments from that surface. Bolt describes the graphical command in procedural terms: a creation routine receives attributes, and the deictic word “there” functions as a temporal signal that samples the pointing cursor. The location is therefore not encoded in the lexical content of the word. The user’s hand supplies a changing coordinate stream, and the speech event selects one instant from it [Bolt, 1980]. In the same example, size can be omitted because the interface



supplies a default value. Omission has several different meanings inside one command. Some information is mandatory in speech, some is supplied by gesture, and some is filled automatically.

The familiar distinction between explicit and implicit prompting is too coarse for this case. A better question is: who or what supplied each operative value? For the square, color and shape can be tagged as language-supplied, position as gesture-supplied, and size as default-supplied. In a contemporary agent system, the same provenance vocabulary would need to expand. A property might be stated by the user, recovered from conversation history, retrieved from a database, inherited from a template, supplied by a tool, defaulted by an interface, inferred by a model, or chosen by an executor. These sources have different implications for attribution and repair. If an unwanted value came from a default, rewriting the user's sentence is an indirect fix. If it came from a retrieved state, the failure belongs elsewhere.

The same distributed structure appears in reference. Bolt's user can act on an object by saying "that" while pointing, without possessing or uttering the system's name for the object. The user can know which object is intended without being able to describe it propositionally [Bolt, 1980]. This matters for generative interfaces because text boxes force two tasks into one channel: the user must both identify a thing and state what relation or operation should apply to it. A directly addressable world can separate those burdens. Pointing, selection, object identity, and spatial persistence can carry reference while language carries relations, intentions, or constraints. In such a system, shorter language need not mean less specification. The missing words may have been replaced by higher-bandwidth state and gesture.

Engelbart's brick-pencil thought experiment gives a historical reason to take these interface effects seriously. A tool can change the visible morphology of symbolic work. When the writing instrument becomes physically awkward, users do not merely work more slowly; under pressure they enlarge their writing and alter their technique [Engelbart, 1962]. Prompt style can be studied in the same way. Repetition, headings, delimiters, defensive restatement, and other conventions may sometimes be durable cognitive aids, but they may also be compensatory forms produced by a particular model-interface relation. A history of prompt technique should therefore ask which linguistic forms survive when the underlying friction disappears.

3 Operative Force Is Assigned at a Commitment Boundary

Distributed specification explains where inputs come from, but it does not yet explain when an inscription becomes operative. McCarthy's LISP evaluator supplies a clean technical contrast. The interior symbolic expression does not determine its own evaluation status. A surrounding form such as QUOTE changes what the evaluator does with the same symbolic material. Under quotation, the expression is taken as itself rather than recursively evaluated; under ordinary evaluation, the expression participates in compu-



tation [McCarthy, 1960]. The causal status of the expression depends on a relation between representation and evaluator.

Natural-language agent systems usually lack a single formal eval operation, but the distinction survives in distributed form. Text can be displayed, stored, summarized, embedded into memory, interpreted as a tool request, routed to an executor, or used to construct another prompt. The same imperative sentence can therefore remain inert in one channel and produce external effects in another. ReAct makes this split explicit inside a language-agent architecture. It augments the environmental action space with a language space. A language-space thought does not itself produce an external observation, but it enters the context that conditions subsequent reasoning and action [Yao et al., 2023]. “Inert” is therefore relative to a state boundary. The thought is externally inert while remaining causally active with respect to the next model state.

This suggests that a binary distinction between text and action is still too simple. A useful causal inventory must name the state that changed. A generated span can modify conversational context without changing a file. It can update memory without invoking a tool. It can cause a tool call without directly producing the tool’s external effect. It can alter a human operator’s decision even if every machine state remains fixed. For agent instrumentation, the relevant question is not simply whether the model “acted.” It is which layer accepted a representation as a commitment and which state spaces changed afterward.

An explicit commitment boundary also clarifies why plain-language containment is fragile. Telling a model to “treat the following as data” is not equivalent to a language runtime’s enforced quotation mode. Both the instruction that establishes the boundary and the supposedly inert content can be visible to the same interpreter. The LISP example shows a different architecture: evaluation status is carried by syntax that the evaluator itself enforces. The corresponding design problem for language-controlled systems is to separate content from execution authority in a channel that the content cannot redefine by persuasion alone. This is an architectural problem, not a prompt-wording trick.

4 Stateful Reference, Constructive Definitions, and Persistent Rules

SHRDLU shows a second way in which the prompt string is too small. Winograd’s system does not interpret each sentence against an empty background. It uses discourse context, world knowledge, a representation of its own actions, and a modeled scene in order to answer questions and execute commands [Winograd, 1971]. In the canonical exchange involving a block taller than the one the robot is holding, the pronoun “it” can be resolved only because the system maintains information about prior action and the current world. Part of what appears to be linguistic competence is therefore state bookkeeping outside the current sentence.

More importantly, SHRDLU can expose some of that bookkeeping before acting. In the



documented dialogue, the system explicitly states what it assumes “it” refers to before proceeding. This is a small but consequential interface choice. The user does not have to infer reference resolution only from the final movement of a block. A fragile binding becomes an inspectable assertion before execution. The design principle is not that every interpretation should be narrated. Constant confirmation would be unusable. The more specific possibility is consequence-sensitive exposure: surface a binding when plausible alternatives would lead to materially different actions.

SHRDLU also complicates the idea that a new prompt concept corresponds to a new capability. A term such as “steeple” can be defined through relations already available in the blocks world and then used in later reasoning and construction. The new noun becomes operative because it composes an existing vocabulary of shapes, sizes, spatial relations, search procedures, and manipulation actions [Winograd, 1971]. A user-defined concept can therefore reorganize capabilities that were already executable. The question “did the prompt add a capability?” is underspecified unless one distinguishes a genuinely new primitive from a new composition of existing operations.

Sutherland’s Sketchpad makes persistence even more concrete. A graphical operation can install a relation whose most important consequence is not visible in the immediate frame. Sutherland describes constraints, attachers, copies, and instances through which a relation such as equal length can be copied and merged into other graphical elements. The scaffolding used to establish the constraint can disappear while the constraint continues to govern future changes [Sutherland, 1963]. The visible delta can be small while the causal delta is large. A one-time edit and a persistent rule can therefore produce the same screen-shot and very different futures.

Sketchpad’s distinction between instances and copies also shows why visual continuity is not enough for persistent worlds. Two shapes can look the same while supporting different future operations. A component can preserve a fixed internal identity while another component remains structurally flexible. Identity is therefore not one Boolean property of the rendered object. It can attach at different scopes: whole object, component, relation, role, or provenance. Language such as “keep the same character but change the costume” presupposes some such layered identity, whether it is represented symbolically or reconstructed by another mechanism. A persistent world must preserve not only appearance but enough addressable structure for later references and transformations to have stable targets.

5 Tests and Evaluators Are Part of the Effective Input

The same distributed analysis applies to evaluation. Pfungst’s Clever Hans experiments remain useful because they show how information can enter a task from the evaluator’s side of an arrangement without anyone intending to send it. In word-reading trials, Hans performed perfectly when the questioner knew the requested placard’s position and failed when the questioner lacked that positional knowledge [Pfungst, 1911]. The nominal vis-



ible task remained a reading task. What changed was a state variable in another participant. The result demonstrated that the effective input included information carried by the experimental arrangement.

For model evaluation, the corresponding question is not whether the grader is honest or blinded in an informal sense. It is which components possess answer-relevant information and which channels can transmit it before commitment. A model can obtain target-correlated evidence from task wrappers, metadata, repeated grader responses, retrieval tools, benchmark conventions, or prior exposure. Capability attribution therefore requires interventions over the whole information topology. A benchmark can only isolate a model property if the alternative channels that could produce the observed performance have been controlled or at least measured.

Program synthesis introduces another evaluation problem. Solar-Lezama and colleagues note that a buggy iterative stencil can still converge to the correct final answer while taking much longer to do so [Solar-Lezama et al., 2007]. A terminal-output verifier can therefore accept an implementation whose process is defective. This is directly relevant to prompt systems with agents and tools. If the test observes only the final artifact, it can miss illegal intermediate states, wasted resource use, brittle dependencies, repeated retries, or policy violations. More verification is not automatically better verification. The verifier must observe variables that correspond to the intended property.

The tension is real because process constraints can also become over-specification. A verifier that dictates every intermediate step prevents alternative valid procedures and defeats part of the value of generative search. The design task is to separate invariants from open implementation choices. In a building system, attachment validity or collision constraints may be true invariants while construction sequence remains open. In an agent, authorization boundaries may be invariant while the internal plan remains flexible. The test should constrain what must remain true, not recreate a full hand-written solution unless the task actually requires one.

6 Repair Changes the Meaning of Accuracy

Bolt's later description of *Put-That-There* explicitly begins from fallibility. The project assumes that speech recognition will not be perfectly accurate and pursues greater "effective accuracy" through redundant channels, context-sensitive interpretation, early feedback, and easy correction [Bolt et al., 1982]. This reframes the engineering problem. A system can become more useful without improving the recognizer if it reduces the distance between a wrong interpretation and the moment at which the user can detect and correct it.

The interface provides several kinds of feedback at different temporal scales. Pointing can be shown continuously as a cursor. A spoken deictic such as "there" commits a sampled location at a particular moment. An addressed graphical item can receive immediate visual



acknowledgment before the larger action completes [Bolt, 1980]. There is no single interpretation moment. Evidence accumulates continuously, some bindings become discrete commitments, and the final world change occurs later. A useful trace should preserve these distinctions rather than collapsing the whole sequence into one prompt and one response.

This produces a different criterion for prompt-interface quality. First-pass task success remains important, but it is incomplete. Systems should also be evaluated by detection latency, repair distance, correction effort, and propagation cost. A weaker model with visible provisional bindings and cheap repair can outperform a stronger opaque model on completed-task correctness. That is an empirical claim to be tested, not assumed, but Bolt’s architecture shows that such a comparison is coherent. It moves research away from the premise that all improvement must come from better linguistic instruction or a more capable recognizer.

Repair also changes the status of a “test.” A system need not wait until the end of a generation to compare output with intent. Referential highlighting, previews, selected tool displays, constraint visualizations, and snap cues can function as micro-tests of intermediate commitments. Each one turns a hidden interpretation into an inspectable state. The appropriate interface need not interrupt the user with a question at every step. It can expose enough state for the user to notice divergence before errors propagate.

7 Control Is Not a Scalar Possession

Ashby’s account of regulation gives a precise reason why prompt specificity should not be confused with control. His automatic-pilot example treats yaw, pitch, and roll as distinct essential variables that must remain within acceptable limits; his bicycle example centers on a different essential variable, the rider’s angle with the vertical [Ashby, 1956]. Control depends on what dimensions must be regulated, what disturbances affect them, what can be observed, and what responses are available. It is not a single quantity that one either possesses or lacks.

A human operator can therefore have formal authority without having adequate regulatory capacity. A stop button can terminate a system while providing almost no ability to steer ordinary behavior. A text prompt can express a high-level preference while exposing none of the intermediate variables required to diagnose a failure. In complex agent systems, “human control” should be decomposed into observable essential variables and available intervention channels. The dimensions should be discovered in relation to a concrete deployment rather than supplied as a generic list.

This is where distributed specification and repair return to control theory. Direct manipulation can move reference work out of prose. Constraint visualizations can expose persistent rules. Commitment previews can reveal selected tools or targets. Process-aware tests can make previously hidden deviations observable. These interface structures increase



the regulator's available distinctions and interventions. They do not guarantee control, but they alter the variety of situations to which the operator can respond. The design of a prompt interface is therefore partly the design of a regulator's observation and action repertoire.

8 Why More Documentation Is Not Enough

Instrumentation can still fail if it is treated as a substitute for understanding. Naur's account of programming as theory building describes a compiler-maintenance case in which a later group possessed extensive program text and documentation but did not recognize that an existing facility applied to a new requirement. The original programmers saw a simpler modification because they possessed a theory of how the system's parts related to its domain [Naur, 1985]. The missing resource was not merely another sentence in the manual. It was a discriminating judgment about which difference was relevant.

Prompt systems invite the same failure mode. A team can accumulate logs, retries, exception clauses, and successful patches while losing the abstraction that made the system easy to modify. Two changes can pass the same local test while only one preserves a reusable account of the mechanism. A useful harness should therefore preserve cases and rationales in a form that supports comparison across superficially different failures. The strongest evidence of theory is not successful repetition of a known recipe but coherent modification under a novel change.

The retracted TeX 0.7 episode preserved in Knuth's error log makes the complementary point. A configuration can return to an earlier textual state after evidence overturns an intervening change, yet the knowledge state of the project has changed. The same active text before and after retraction does not imply the same theory of why the text is correct [Knuth, 1992]. For prompt harnesses, deleted clauses and failed fixes can be valuable negative knowledge. Versioning should be able to preserve the evidence that introduced and later killed a rule, rather than storing only the currently active prompt.

9 Prompt as a Family of Operative Arrangements

These cases do not imply that every event called prompting has the same hidden architecture. Wittgenstein's discussion of games provides a better methodological model. He moves from board games to card games, patience, ball games, and the child throwing a ball against a wall, watching candidate common properties appear and disappear rather than insisting on one feature present in every case [Wittgenstein, 1953]. The useful operation is controlled comparison between neighbors.

Prompt research can use the same strategy. Begin with a paradigmatic agentic natural-language prompt and remove one feature at a time. Remove the explicit verifier while



holding the executor fixed. Replace text with gesture while preserving the operation. Replace a stochastic model with a deterministic program. Remove persistent world state. Remove the user-authored move and let an agent initiate the action. At each transition, ask what explanatory work the word “prompt” still performs. This approach can reveal overlapping operational families without turning family resemblance into an excuse for vagueness.

The result also limits the thesis of this paper. Distributed specification, commitment boundaries, and repair are not proposed as necessary and sufficient conditions for every prompt. They are recurring structures that become visible in historically and technically specific systems. Some interfaces will have no gesture channel. Some operations will have no persistent world. Some model calls will be genuinely one-shot. The claim is narrower: when those structures are present, treating the visible string as the complete causal or semantic unit discards information needed to explain the result.

10 An Instrumentation Agenda

A practical research program follows from the distinctions above. First, systems should record parameter provenance where it can be represented without pretending to more precision than the architecture supports. If a value came from a user utterance, a pointer sample, a default, a retrieved record, a persistent object identity, or an executor policy, the trace should distinguish those sources. For neural generation, provenance will often be coarser than a constructor’s argument list, but even coarse provenance is more informative than attributing every property to the prompt text.

Second, interfaces should distinguish evidence from commitment. Continuously changing evidence can remain provisional; a tool selection, object binding, coordinate sample, permission grant, or transaction commit can be logged as a discrete state transition. Consequential bindings should be exposed before execution when plausible alternatives would lead to materially different effects. This is not a demand for constant confirmation. It is a demand that the architecture know where commitment occurs and give the interface the option to surface it.

Third, traces should identify the state space affected by every operation. ReAct’s thought/action distinction is one example, but a contemporary agent may also update memory, retrieval state, tool state, file state, environment state, or human decision state. “No action” is meaningful only relative to one of these spaces. A causal-state inventory can prevent a system from labeling an operation inert simply because it did not immediately touch the external environment.

Fourth, tests should be attached to the property being claimed. Terminal output is appropriate for some tasks, but it is insufficient when trajectory, resource use, authorization, persistent constraints, or provenance are part of the specification. Conversely, tests should not unnecessarily freeze implementation choices that should remain open. The



relevant distinction is between invariants and degrees of freedom, not between strict and loose prompting.

Fifth, repair should be measured. First-pass success obscures interaction systems whose mistakes are cheap to see and correct, as well as systems whose rare mistakes propagate catastrophically before they become visible. Detection latency, repair distance, correction burden, and residual state after correction are all observable quantities. These measures move prompt evaluation closer to the actual work of operating a system over time.

Finally, archives should preserve failed theories as well as successful configurations. Naur's maintenance case and Knuth's retracted version both show why a final artifact cannot carry the complete history of the judgments that produced it. Prompt harnesses that record only the latest system message will repeatedly rediscover old mistakes. A useful research record should preserve the failure that motivated a change, the mechanism the change was supposed to address, the evidence that supported it, and the counterevidence that later invalidated it.

11 Limitations and Unresolved Territory

The cases assembled here differ sharply in scale and mechanism. Sketchpad and LISP expose deterministic structures that contemporary neural systems often hide. SHRDLU works in a deliberately tiny symbolic world. Clever Hans concerns perceptible cues in an animal experiment. Ashby's variables are analytical abstractions, not ready-made metrics for AI governance. ReAct describes a research architecture rather than every deployed agent. These differences block any simple historical claim that modern prompting directly descends from one of these systems. The comparisons are functional and diagnostic unless documentary evidence establishes a genealogy.

The central unresolved technical problem is representation. Parameter provenance is easy when a constructor has explicit arguments and difficult when a generated property emerges from distributed latent computation. Commitment is easy to mark when a transaction has a clear commit operation and harder when model output incrementally affects downstream processes. Persistent identity is explicit in Sketchpad and often heuristic in generated images or worlds. The paper therefore proposes instrumentation targets without assuming that every target has a clean symbolic implementation.

A second unresolved problem is the boundary between helpful exposure and interface overload. Making every dependency visible can produce an unusable display. Asking for confirmation at every ambiguity can destroy fluent interaction. Process-aware verification can overconstrain search. History-sensitive feedback can make a system difficult to learn. The design question is selective exposure: which hidden states have sufficiently different downstream consequences that the user should see them before commitment?

A third problem concerns the term "prompt" itself. If future systems move reference into selection, constraints into direct manipulation, evaluation into executable tests, and del-



egration into persistent agents, the visible prompt may shrink while the surrounding operative arrangement becomes richer. The category could stabilize around high-level underspecified instruction, or it could dissolve into a family of specialized controls. The appropriate research response is not to protect the word. It is to preserve the distinctions needed to explain how descriptions become operations in each concrete practice.

12 Conclusion

The sentence is often the most visible part of a prompt system, but visibility is not causal sufficiency. Bolt's square receives properties from speech, gesture, and defaults. LISP separates symbolic content from evaluation status. SHRDLU resolves reference against persistent world and discourse state and can make a binding explicit before action. Sketchpad installs rules that survive beyond the visible edit. ReAct separates changes to context from changes to environment. Pfungst shows that information on the evaluator's side can become effective input. Solar-Lezama shows that a correct terminal answer can conceal a defective process. Ashby shows why control depends on the dimensions that can be observed and regulated. Naur shows that even extensive traces and documentation do not automatically yield the theory needed for modification.

Taken together, these cases support a concrete change in research practice. Stop asking only what words were sent to the model. Ask where each operative value came from, when an interpretation became a commitment, which state changed, what feedback exposed the change, what test evaluated it, and how far an error traveled before repair. That expanded trace does not eliminate the prompt. It places the prompt inside the language-game that gives it force.

Acknowledgment of Process

This working paper was assembled in an AI-augmented research process from preserved zettels and cited sources. The immutable zettel payloads, prompts, source map, making history, and rebuild path are preserved in the accompanying SLIPCASE package. No human scholarly review is claimed for this draft.

References

- W. Ross Ashby. *An Introduction to Cybernetics*. Chapman & Hall, London, 1956.
- Richard A. Bolt. "Put-that-there": Voice and Gesture at the Graphics Interface. In *Proceedings of SIGGRAPH 1980*, pages 262–270. ACM, 1980. doi: 10.1145/800250.807503.



- Richard A. Bolt, Chris Schmandt, and Eric A. Hulteen. Put-that-there: Voice and gesture at the graphics interface. 1982. Architecture Machine Group, Massachusetts Institute of Technology.
- Douglas C. Engelbart. Augmenting human intellect: A conceptual framework. Technical Report AFOSR-3223, Stanford Research Institute, 1962.
- Donald E. Knuth. *Literate Programming*. Center for the Study of Language and Information, 1992.
- John McCarthy. Recursive functions of symbolic expressions and their computation by machine, part i. *Communications of the ACM*, 3(4):184–195, 1960.
- Peter Naur. Programming as theory building. *Microprocessing and Microprogramming*, 15: 253–261, 1985.
- Oskar Pfungst. *Clever Hans (The Horse of Mr. von Osten): A Contribution to Experimental Animal and Human Psychology*. Henry Holt and Company, New York, 1911.
- Armando Solar-Lezama, Gilad Arnold, Liviu Tancau, Rastislav Bodik, Vijay Saraswat, and Sanjit Seshia. Sketching stencils. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2007.
- Ivan Edward Sutherland. *Sketchpad: A Man-Machine Graphical Communication System*. PhD thesis, Massachusetts Institute of Technology, 1963.
- Terry Winograd. Procedures as a representation for data in a computer program for understanding natural language. Technical Report AI-TR-235, MIT Artificial Intelligence Laboratory, 1971.
- Ludwig Wittgenstein. *Philosophical Investigations*. Basil Blackwell, Oxford, 1953.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

A Assembly Instrument

The exact publication assembly prompt is preserved at `_PROMPTS/SLIPCASE_ASSEMBLY_PROMPT_15_55_AM.txt` in the accompanying package and embedded in `index.html`. The full prompt is omitted here only to keep the working paper readable.



B Making History

PROVIDED: the research prompts and 41-page proposal deck. PRESERVED: the admitted immutable zettel payloads, prompt bodies, and local uploaded PDF. RETRIEVED: public source pages used for source verification during assembly. DERIVED: this paper, source map, bibliography views, graph, MOCs, arrangements, network, reader, mark, and verification report. VERIFIED: deterministic hashes and structural counts; PDF compilation and rendering are recorded in the package verification report. UNVERIFIED: complete conversation coverage and human scholarly review.

C Replication Path

Open `index.html` offline to inspect the admitted cards, sources, bibliography, prompts, ghosts, and paper metadata. Use `000__RETURN_PATH.txt` to rejoin the checkpoint. Use `ZETTELS.json` and `_SLIPCASE/reconstruct_cards.py` to reconstruct immutable card files and verify their SHA-256 identities. Full limitations are recorded in `_SLIPCASE/VERIFICATION.txt`.

